



CPE3243 วิศวกรรมซอฟต์แวร์ (Software Engineering)

Piyavit Laung-Aram
Major of Computer Engineering
Faculty of Engineering
Ramkhamhaeng University, Thailand

การเขียนโปรแกรมด้วยภาษา Java ส่วนที่ 4

(Java Programming Part 4)



Java Method

- เมธอดคือบล็อกของโค้ดที่จะทำงานเมื่อมีการเรียกใช้เท่านั้น
- คุณสามารถส่งข้อมูลหรือผ่านพารามิเตอร์ไปยังเมธอดได้
- เมธอดใช้เพื่อดำเนินการบางอย่าง และเรียกอีกอย่างว่าฟังก์ชัน
- ที่ต้องใช้เมธอด เนื่องจากหากต้องการใช้โค้ดซ้ำ เราสามารถเขียนโค้ดเพียงครั้งเดียว และเรียกใช้หลายครั้งได้



Java Method(2)

- myMethod คือชื่อของเมธอด
- static หมายความว่าเมธอดนี้เป็นของคลาส Main ไม่ใช่โอब्เจกต์ของคลาส Main
- void หมายความว่าเมธอดนี้ไม่มีค่าส่งกลับ

```
1 public class Main
2 {
3     static void myMethod()
4     {
5         // code
6     }
7 }
```



Java Method(3)

การเรียกใช้เมธอด

- หากต้องการเรียกเมธอดใน Java ให้เขียนชื่อของเมธอดตามด้วยวงเล็บสองตัว () และเครื่องหมายอัฒภาค
- ในตัวอย่างต่อไปนี้ myMethod() ใช้ในการพิมพ์ข้อความ (การดำเนินการ) เมื่อมันถูกเรียกว่า:

```
//Example54_1.java
public class Main {
    static void myMethod() {
        System.out.println("Hello Thailand");
    }

    public static void main(String[] args) {
        myMethod();
    }
}
```

ผลการรันโปรแกรม

Hello Thailand



Java Method Parameters

พารามิเตอร์และอาร์กิวเมนต์

- ข้อมูลสามารถส่งผ่านไปยังเมธอดผ่านพารามิเตอร์ในเมธอดได้ พารามิเตอร์ทำหน้าที่เป็นตัวแปรภายในเมธอด
- พารามิเตอร์จะถูกระบุหลังชื่อวิธีการ ภายในวงเล็บ คุณสามารถเพิ่มพารามิเตอร์ได้มากเท่าที่คุณต้องการ เพียงแค่ใช้เครื่องหมายจุลภาค
- ตัวอย่างต่อไปนี้มีวิธีการที่ใช้สตริงที่เรียกว่า fname เป็นพารามิเตอร์ เมื่อเรียกใช้เมธอด เราจะส่งต่อชื่อ ซึ่งใช้ในเมธอดเพื่อพิมพ์ชื่อเต็ม



Java Method Parameters(2)

- ค่าที่ถูกส่งผ่านพารามิเตอร์ไปยังเมธอด เราจะเรียกว่าอาร์กิวเมนต์ จากตัวอย่างข้างต้น fname คือพารามิเตอร์ ในขณะที่ Rodger, Jenny และ Lisa เป็นอาร์กิวเมนต์

```
//Example54_2.java
public class Main {
    static void myMethod(String fname) {
        System.out.println(fname + " Engineer ");
    }

    public static void main(String[] args) {
        myMethod("Rodger");
        myMethod("Jenny");
        myMethod("Lisa");
    }
}
```

ผลการรันโปรแกรม

```
Rodger Engineer
Jenny Engineer
Lisa Engineer
```



Java Method Parameters(3)

Multiple Parameters

```
//Example54_3.java
public class Main {
    static void myMethod(String fname, int age) {
        System.out.println(fname + " is " + age);
    }

    public static void main(String[] args) {
        myMethod("Rodger", 5);
        myMethod("Jenny", 8);
        myMethod("Lisa", 31);
    }
}
```

ผลการรันโปรแกรม

```
Rodger is 5
Jenny is 8
Lisa is 31
```




Java Method Parameters(4)

Return Values

```
//Example54_4.java
public class Main {
    static int myMethod(int x) {
        return 5 + x;
    }

    public static void main(String[] args) {
        System.out.println(myMethod(5));
    }
}
```

ผลการรันโปรแกรม

10

```
//Example54_5.java
public class Main {
    static int myMethod(int x, int y) {
        return x + y;
    }

    public static void main(String[] args) {
        int z = myMethod(10, 5);
        System.out.println(z);
    }
}
```

ผลการรันโปรแกรม

15



Java Method Overloading

Method Overloading คือ หลายเมธอดสามารถมีชื่อเดียวกันแต่มีพารามิเตอร์ต่างกันได้

```
int myMethod(int x)
float myMethod(float x)
double myMethod(double x, double y)
```

```
//Example54_6.java
public class Main {
    static int plusMethod(int x, int y) {
        return x + y;
    }

    static double plusMethod(double x, double y) {
        return x + y;
    }

    public static void main(String[] args) {
        int myNum1 = plusMethod(8, 5);
        double myNum2 = plusMethod(4.3, 6.26);
        System.out.println("int: " + myNum1);
        System.out.println("double: " + myNum2);
    }
}
```

ผลการรันโปรแกรม

```
int: 13
double: 10.559999999999999
```



Java Scope

ใน Java ตัวแปรสามารถเข้าถึงได้ภายในขอบเขตที่สร้างขึ้นเท่านั้น สิ่งนี้เรียกว่า Scope

Method Scope

ตัวแปรที่ประกาศโดยตรงภายในเมธอดนั้นสามารถใช้ได้ทุกที่ในเมธอดที่อยู่ต่อจากบรรทัดโค้ดที่ถูกประกาศ

```
//Example54_7.java
public class Main {
    public static void main(String[] args) {

        // Code here cannot use x

        int x = 100;

        // Code here can use x
        System.out.println(x);
    }
}
```

ผลการรันโปรแกรม

100



Java Scope(2)

Method Scope

ตัวแปรที่ประกาศโดยตรงภายในเมธอดนั้นสามารถใช้ได้ทุกที่ในเมธอดที่อยู่ต่อจากบรรทัดโค้ดที่ถูกประกาศตัวแปร

```
//Example54_7.java
public class Main {
    public static void main(String[] args) {

        // Code here cannot use x ←
        int x = 100;

        // Code here can use x ←
        System.out.println(x);
    }
}
```

ผลการรันโปรแกรม

100



Java Scope(3)

```
//Example54_8.java
public class Main {
    public static void main(String[] args) {

        // Code here CANNOT use x

        { // This is a block

            // Code here CANNOT use x

            int x = 100;

            // Code here CAN use x
            System.out.println(x);

        } // The block ends here

        // Code here CANNOT use x

    }
}
```

Block Scope

- บล็อกของโค้ดหมายถึงโค้ดทั้งหมดที่อยู่ระหว่างเครื่องหมายปีกกา {}
- ตัวแปรที่ประกาศภายในบล็อกของโค้ดสามารถเข้าถึงได้โดยโค้ดระหว่างเครื่องหมายปีกกาเท่านั้น ซึ่งอยู่ต่อจากบรรทัดที่มีการประกาศตัวแปร



Java Recursion

Java Recursion

- การเรียกซ้ำเป็นเทคนิคในการเรียกใช้ฟังก์ชันด้วยตัวมันเอง เทคนิคนี้มีวิธีแบ่งปัญหาที่ซับซ้อนออกเป็นปัญหาง่ายๆ ซึ่งแก้ไขได้ง่ายกว่า
- การเรียกซ้ำอาจจะเข้าใจยากสักหน่อย วิธีที่ดีที่สุดในการทราบว่ามันทำงานอย่างไรคือการทดลองกับมัน

ตัวอย่างการเรียกซ้ำ

- การบวกตัวเลขสองตัวเข้าด้วยกันนั้นทำได้ง่าย แต่การบวกช่วงตัวเลขนั้นซับซ้อนกว่า ในตัวอย่างต่อไปนี้ การเรียกซ้ำจะใช้เพื่อเพิ่มช่วงของตัวเลขเข้าด้วยกันโดยการแบ่งย่อยออกเป็นงานง่ายๆ ในการบวกตัวเลขสองตัว:



Java Recursion(2)

ใช้การเรียกซ้ำเพื่อเพิ่มตัวเลขทั้งหมดไม่เกิน 10

```
//Example54_9.java
public class Main {
    public static void main(String[] args) {
        int result = sum(10);
        System.out.println(result);
    }
    public static int sum(int k) {
        if (k > 0)
        {
            return k + sum(k - 1);
        }
        else
        {
            return 0;
        }
    }
}
```

ผลการรันโปรแกรม

55

10 + sum(9)
10 + (9 + sum(8))
10 + (9 + (8 + sum(7)))
...
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + sum(0)
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + 0



Halting Condition

สภาพการหยุดชะงัก-Halting Condition

เช่นเดียวกับที่ลูบสามารถประสบปัญหาการวนซ้ำแบบไม่มีที่สิ้นสุด ฟังก์ชันแบบเรียกซ้ำก็สามารถประสบปัญหาการเรียกซ้ำแบบไม่มีที่สิ้นสุดได้ การเรียกซ้ำแบบไม่มีที่สิ้นสุดคือเมื่อฟังก์ชันไม่เคยหยุดเรียกตัวเอง ทุกฟังก์ชันแบบเรียกซ้ำควรมีเงื่อนไขการหยุดซึ่งเป็นเงื่อนไขที่ฟังก์ชันหยุดเรียกตัวเอง ในตัวอย่างก่อนหน้านี้ เงื่อนไขการหยุดคือเมื่อพารามิเตอร์ k กลายเป็น 0

การดูตัวอย่างต่างๆ ที่หลากหลายจะเป็นประโยชน์เพื่อให้เข้าใจแนวคิดได้ดีขึ้น ในตัวอย่างนี้ ฟังก์ชันจะเพิ่มช่วงของตัวเลขระหว่างจุดเริ่มต้นและจุดสิ้นสุด เงื่อนไขการหยุดสำหรับฟังก์ชันแบบเรียกซ้ำนี้คือเมื่อจุดสิ้นสุด-end ไม่มากกว่าจุดเริ่มต้น-start



Halting Condition(2)

```
//Example54_10.java
public class Main {
    public static void main(String[] args) {
        int result = sum(5, 10);
        System.out.println(result);
    }
    public static int sum(int start, int end) {
        if (end > start) {
            return end + sum(start, end - 1);
        } else {
            return end;
        }
    }
}
```

ผลการรันโปรแกรม

45



Java Constructors

Java Constructors

Constructor ใน Java เป็นวิธีการพิเศษที่ใช้ในการเริ่มต้นออบเจกต์ Constructor จะถูกเรียกเมื่อมีการสร้างออบเจกต์ของคลาส สามารถใช้เพื่อตั้งค่าเริ่มต้นสำหรับแอตทริบิวต์ของ ออบเจกต์

```
//Example54_11.java
// Create a Main class
public class Main {
    int x;

    // Create a class constructor for the Main class
    public Main() {
        x = 5;
    }

    public static void main(String[] args) {
        Main myObj = new Main();
        System.out.println(myObj.x);
    }
}
```

ผลการรันโปรแกรม

5



Java Constructors(2)

โปรดทราบว่าชื่อ Constructor ต้องตรงกับ
ชื่อคลาส และไม่สามารถมีประเภทการ
ส่งคืนได้ (เช่น void)

โปรดทราบว่า Constructor จะถูกเรียกเมื่อมีการ
สร้างออบเจกต์

คลาสทั้งหมดมี Constructor เป็นค่าเริ่มต้น
หากคุณไม่ได้สร้าง Constructor ของ
คลาสด้วยตัวเอง Java จะสร้างคลาสขึ้นมา
ให้คุณ อย่างไรก็ตาม คุณจะไม่สามารถตั้ง
ค่าเริ่มต้นสำหรับแอตทริบิวต์ของวัตถุได้

```
//Example54_11.java
// Create a Main class
public class Main {
    int x;

    // Create a class constructor for the Main class
    public Main() {
        x = 5;
    }

    public static void main(String[] args) {
        Main myObj = new Main();
        System.out.println(myObj.x);
    }
}
```



Java Constructors(3)

Constructor Parameters

คอนสตรัคเตอร์ยังสามารถรับค่าผ่านพารามิเตอร์ได้ ซึ่งใช้ในการเริ่มต้นแอตทริบิวต์

ตัวอย่างต่อไปนี้จะเพิ่มพารามิเตอร์ `int y` ให้กับ Constructor ภายในตัวสร้างเราตั้งค่า `x` เป็น `y` (`x=y`) เมื่อเราเรียก Constructor เราจะส่งพารามิเตอร์ไปยัง Constructor (5) ซึ่งจะตั้งค่าของ `x` เป็น 5:

```
//Example54_1/.java
public class Main {
    int x;

    public Main(int y) {
        x = y;
    }

    public static void main(String[] args) {
        Main myObj = new Main(5);
        System.out.println(myObj.x);
    }
}
```

ผลการรันโปรแกรม

5



Java Constructors(4)

- คุณสามารถมีพารามิเตอร์ได้มากเท่าที่คุณต้องการ

```
//Example54_13.java
//filename: Main.java
public class Main {
    int modelYear;
    String modelName;

    public Main(int year, String name) {
        modelYear = year;
        modelName = name;
    }

    public static void main(String[] args) {
        Main myCar = new Main(1969, "Mustang");
        System.out.println(myCar.modelYear + " " +
myCar.modelName);
    }
}
```

ผลการรันโปรแกรม

1969 Mustang



Java Modifiers

```
public class Main
```

- คีย์เวิร์ด public คือตัวแก้ไขการเข้าถึง ซึ่งหมายความว่าใช้เพื่อกำหนดระดับการเข้าถึงสำหรับคลาส คุณลักษณะ วิธีการ และตัวสร้าง

เราแบ่งตัวแก้ไขออกเป็นสองกลุ่ม

- Access Modifiers - ควบคุมระดับการเข้าถึง
- Non-Access Modifiers - ไม่ได้ควบคุมระดับการเข้าถึง แต่มีฟังก์ชันอื่น ๆ ควบคุม



Java Modifiers(2)

Access Modifiers สำหรับ class : คุณสามารถใช้ public เป็นค่าเริ่มต้น

Modifier	Description
public	คลาสอื่นสามารถเข้าถึงได้

```
//Example54_14.java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

ผลการรันโปรแกรม

Hello World

default	คลาสสามารถเข้าถึงได้โดยคลาสในแพ็คเกจเดียวกันเท่านั้น ใช้เมื่อคุณไม่ได้ระบุตัว Modifier
---------	---

```
//Example54_15.java
class MyClass {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

ผลการรันโปรแกรม

Hello World



Java Modifiers(3)

Access Modifiers สำหรับ attributes, methods and constructor

Modifier	Description
public	คลาสอื่นสามารถเข้าถึง โค้ดนี้ได้ทุกคลาส

ผลการรันโปรแกรม

Name: John DoDoDo
Email: john@dododo.com
Age: 24

```
//Example54_16.java
public class Main {
    public String fname = "John";
    public String lname = "DoDoDo";
    public String email = "john@dododo.com";
    public int age = 24;
}

class Second {
    public static void main(String[] args) {
        Main myObj = new Main();
        System.out.println("Name: " + myObj.fname + " " + myObj.lname);
        System.out.println("Email: " + myObj.email);
        System.out.println("Age: " + myObj.age);
    }
}
```




Java Modifiers(4)

Access Modifiers สำหรับ **attributes, methods and constructor**

Modifier	Description
----------	-------------

private	โค้ดสามารถเข้าถึงได้จากภายในคลาสที่ประกาศเท่านั้น
---------	---

```
//Example54_17.java
public class Main {
    private String fname = "John";
    private String lname = "DoDoDo";
    private String email = "john@dododo.com";
    private int age = 24;

    public static void main(String[] args) {
        Main myObj = new Main();
        System.out.println("Name: " + myObj.fname + " " +
myObj.lname);
        System.out.println("Email: " + myObj.email);
        System.out.println("Age: " + myObj.age);
    }
}
```

ผลการรันโปรแกรม

```
Name: John DoDoDo
Email: john@dododo.com
Age: 24
```



Java Modifiers(4)

Access Modifiers สำหรับ **attributes, methods and constructor**

Modifier	Description
----------	-------------

default	โค้ดสามารถเข้าถึงได้เฉพาะในแพ็คเกจเดียวกันเท่านั้น ใช้เมื่อไม่ได้ Modifier
---------	---

```
//Example54_18.java
class Person {
    String fname = "John";
    String lname = "DoDoDo";
    String email = "john@dododo.com";
    int age = 24;

    public static void main(String[] args) {
        Person myObj = new Person();
        System.out.println("Name: " + myObj.fname + " " +
myObj.lname);
        System.out.println("Email: " + myObj.email);
        System.out.println("Age: " + myObj.age);
    }
}
```

ผลการรันโปรแกรม

```
Name: John DoDoDo
Email: john@dododo.com
Age: 24
```



Java Modifiers(4)

Access Modifiers สำหรับ **attributes, methods and constructor**

Modifier	Description
----------	-------------

protected	โค้ดสามารถเข้าถึงได้ในแพ็คเกจและคลาสย่อยเดียวกัน
-----------	--

```
//Example54_19.java
class Person {
    protected String fname = "John";
    protected String lname = "DoDoDo";
    protected String email = "john@dododo.com";
    protected int age = 24;
}

class Student extends Person {
    private int graduationYear = 2018;
    public static void main(String[] args) {
        Student myObj = new Student();
        System.out.println("Name: " + myObj.fname + " " +
myObj.lname);
        System.out.println("Email: " + myObj.email);
        System.out.println("Age: " + myObj.age);
        System.out.println("Graduation Year: " +
myObj.graduationYear);
    }
}
```

ผลการรันโปรแกรม

```
Name: John DoDoDo
Email: john@dododo.com
Age: 24
```



พบกันตอนต่อไป



สามารถสอบถามได้ครับ

Q & A